

D) Aim:

To design and implement a program using recursive descent parsing for the grammar $S \rightarrow aA$, $A \rightarrow bA | c$ and verify the input $abbc$.

ALGORITHM:

- * Start the parsing with start symbol
- * Check and match a according to the rule $S \rightarrow aA$.
- * Call function $A()$ after matching a !
- * Match b recursively or c according to $A \rightarrow bA | c$
- * If the complete input is consumed, display - accepted ; Otherwise display Rejected.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
char s[20]; int i=0;
void A() {
    if (s[i] == 'b') {i++; A();}
    else if (s[i] == 'c') i++;
}
```

```
void $11 {
    if (s[i] == 'a') { i++; A(); }
}
```

```
int main() {
    scanf ("%s", s);
```

```
    s();
    printf (i == strlen(s) ? "Accepted\n" : "Rejected\n");
```

```
    return 0; }
```

Output:

Input: obbc

Accepted

RESULT:

Thus, the input obbc is successfully accepted by the recursive descent parser.

2) Aim:

To design and implement a C program that converts three-address code into assembly-like target instructions.

ALGORITHM:

- * Read the given three - Address code statements.
- * Identify the arithmetic operations in each statement.
- * Load the required operands into a register.
- * Perform the corresponding arithmetic operation and store the result.
- * Display the generated assembly - like target instructions.

PROGRAM:

```
# include <stdio.h>
```

```
int main()
```

```
{  
    printf("LOAD R1, a\n");  
    printf("ADD R1, b\n");  
    printf("STORE t1, R1\n");  
    printf("LOAD R1, t1\n");  
    printf("SUB R1, c\n");  
    printf("STORE t2, R1\n");  
    printf("LOAD R1, t2\n");  
}
```

```
printf("STORE x, R, ln");  
return 0;
```

}

Output:

LOAD R1, a

ADD R1, b

STORE t1, R1

LOAD R1, t1

SUB R1, c

STORE t2, R1

LOAD R1, t2

STORE x, R1

RESULT:

Thus, the given Three - Address code is successfully converted into assembly like target instructions.

3)

Aim:

To write a LEX program to check whether a given symbol address is valid or invalid.

ALGO RITHM:

- * Read an email address as input
- * Define a regular expression for a valid email format.
- * Match the input with the defined pattern.
- * Display Valid Email if the pattern matches.
- * Otherwise, display Invalid Email.

PROGRAM:

```
% {  
#include <stdio.h>  
%}  
%  
[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]+  
printf("Valid Email\n");  
• printf("Invalid Email\n");  
%  
%  
int main() { yylex(1);  
return yywrap(); }
```

Output:

Input: student@gmail.com

Valid Email

RESULT:

Thus, the LEX program successfully checks whether an email address is valid or invalid.

4)

Aim:

To write a LEX program to identify whether the given integers are even or odd.

ALGORITHM:

- * Read the integer from the input.
- * Match the integer using a regular expression.
- * Check its last digit.
- * If the last digit is 0, 2, 4, 6 or 8, identify it as even.
- * Otherwise, identify it as odd and display the result.

PROGRAM:

```
% {  
#include <stdio.h>  
  
% }  
% }  
[0-9] * [02468] print f ("Even\n");  
[0-9] * [13579] print f ("Odd\n");  
[!t\n]+;  
% }  
int main() { yylex(); }  
int yywrap() { return 1; }
```

OUTPUT:

Input : 12

Even .

Input : 25

Odd

RESULT:

Thus the .lex program successfully identifies the given integers as even or odd.